

Installing MessageFoundry — the engine + your configuration repository

Early Access · MessageFoundry v0.3.2 · July 2026

This guide explains how an organization installs **MessageFoundry (MEFOR)** and stands up the **private git repository** that holds its integration configuration. It is the practical companion to [ADR 0017](#) (the consumer deployment model).

Scope. This guide covers *install + config-repo + private git + running multiple instances*. For the staged, go/no-go-gated path from a first install to full production (lab → shadow → limited → full), plus security/PHI hardening, reliability tuning, and DR, read the **Early-Adopter Installation & Rollout Guide**. For network exposure / TLS specifics see **DEPLOYMENT.md**; for the Windows service see **SERVICE.md**.

1. The model in one paragraph

MessageFoundry ships as a **read-only, version-pinned engine** (a Python wheel) that your team installs but **never edits**. All of *your* work — the Connections, Routers, and Handlers that define your interfaces — lives in a **separate, private git repository your organization owns**: your *config repo*. One config repo drives **all** of your engine instances (Test and Production at minimum; optionally POC/Staging), with each instance selecting its environment and security posture at runtime. The practical consequence: integration developers author and review interface logic in a normal git/PR workflow, and the engine underneath them is a fixed, auditable dependency they cannot quietly modify.

Three ownership tiers — keeping them separate is the whole design:

Tier	What it is	Who owns / edits it
Engine	The installed <code>messagefoundry</code> wheel (pinned version)	The MessageFoundry project — you install it, never edit it
Config repo	Your private git repo: Connection/Router/Handler <code>.py</code> , code sets, <code>environments/<env>.toml</code> , fixtures	Your integration developers — authored via pull requests
Per-instance settings	<code>messagefoundry.toml</code> + <code>MEFOR_*</code> environment variables + secrets	Your operators, per deployed instance — never committed to the repo

2. Prerequisites

- **Python 3.14+** on each engine host (the engine requires 3.14+).
- **git**, plus a **private git host** — GitHub (private repo), GitLab, Azure DevOps, Bitbucket, or a self-hosted server. Nothing about MessageFoundry requires a public repo; your config repo is yours.
- A source for the engine wheel: **public PyPI** is the target distribution channel; until the project publishes its first release, the signed **GitHub Release wheel** or an **internal package index** (Artifactory, Azure Artifacts, a private PyPI) serves the same role.
- Administrator/elevation on the host if you will install the engine as a Windows service (see [SERVICE.md](#)).

3. Step 1 — Install the engine (a pinned, read-only dependency)

On each host, create a virtual environment and install the engine at a **pinned version**:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # Linux/macOS: . .venv/bin/activate
pip install "messagefoundry==0.3.2"  # pin the exact version (core runtime only)
```

⚠ Early access. MessageFoundry is in **Early Access** on public PyPI — feature-complete and test-validated, but the independent external review and penetration test that ASVS recommends at Level 3 have not yet been performed. Pin the version you have qualified; check [PyPI](#) for the current release. You can equally install from your organization's **private index**.

Add extras only for what a host actually runs — `messagefoundry[postgres]` (PostgreSQL store), `messagefoundry[sqlserver]` (SQL Server store + the DATABASE connectors, needs OS-level ODBC Driver 18), `messagefoundry[harness]` (the PySide6 test harness GUI), `messagefoundry[sftp]` (SFTP connectors), `messagefoundry[fhir]` (FHIR codec + FHIR outbound), `messagefoundry[dicom]` (DICOM C-STORE SCP + codec).

Key properties this install model gives you:

- **Non-editable.** A normal `pip install` lands the engine in `site-packages` as a regular installed package — not an editable checkout. Developers import its public surface; they do not have the engine source tree in front of them to change.
- **Pinned + reproducible.** The exact version is recorded in your config repo's `requirements.txt` (Step 2). An engine upgrade is a deliberate, reviewable one-line bump — never an accident. For a fully reproducible deploy, extend that into a hash-locked requirements file and install with `--require-hashes`.

Verify the release before you install (supply-chain integrity)

MessageFoundry ships **one signed wheel to many PHI-bearing instances**, so verify the artifact's provenance *before* installing it — pinning a version (or a hash) proves you got a *fixed* file, not that it's the one MessageFoundry built. Every release carries **SLSA build provenance** (binding the wheel's SHA-256 → the source commit → the GitHub Actions builder) and a **Sigstore signature**. Check both with the **GitHub CLI** (`gh ≥ 2.49`), and optionally `sigstore` (`pip install sigstore`); install **only** the file that passes.

```
$V = "0.3.2" # the version you intend to install – see PyPI for the current release

# Download the wheel + its Sigstore bundle from that release's assets
gh release download "v$V" --repo MEFORORG/MessageFoundry `
  --pattern "messagefoundry-$V-*.whl" --pattern "messagefoundry-$V-*.whl.sigstore*"

# Verify SLSA build provenance: artifact -> source commit -> builder workflow
gh attestation verify "messagefoundry-$V-py3-none-any.whl" --repo MEFORORG/MessageFoundry

# (defense in depth) Verify the Sigstore signature pins the release workflow identity
python -m sigstore verify identity "messagefoundry-$V-py3-none-any.whl" `
  --cert-identity "https://github.com/MEFORORG/MessageFoundry/.github/workflows/release.yml@refs/tags/v$V"
  --cert-oidc-issuer "https://token.actions.githubusercontent.com"

# Only if BOTH pass, install the exact file you verified
pip install ".\messagefoundry-$V-py3-none-any.whl"
```

The same attestation also covers the **public PyPI** copy of the wheel (it is byte-identical to the GitHub-built artifact, so the digest matches), so you can download-verify-then-install from the index:

```
$V = "0.3.2"
pip download "messagefoundry==$V" --no-deps -d .\verify
gh attestation verify (Get-ChildItem ".\verify\messagefoundry-$V-*.whl").FullName --repo
MEFORORG/MessageFoundry
pip install --no-index --find-links .\verify "messagefoundry==$V"
```

A registry/mirror substitution or a relabelled file **fails** the check. For a fully pinned deploy, pair this with `pip install --require-hashes -r requirements.lock` (the identity check above is the part `--require-hashes` cannot give you — it proves bytes-match-lockfile, not who built them). The `-rc`-tagged pre-releases publish to production PyPI too; the `--cert-identity` ref above must match the tag you are installing (e.g. `refs/tags/v0.3.2`).

4. Step 2 — Create your config repo with the scaffolder

The engine ships an `init` command that lays down a complete, **check -green-from-the-first-commit** config repo skeleton:

```
messagefoundry init ./my-config-repo
cd my-config-repo
```

It writes:

```
my-config-repo/
├─ config/IB_EXAMPLE_ADT.py      # a runnable starter feed (receive ADT over MLLP → archive to file)
├─ environments/dev.toml        # non-secret per-environment values for env("...") lookups
└─ environments/prod.toml
```

```

└─ messages/sets/example_adt.hl7 # a synthetic fixture (NO real PHI) that gates `check`
└─ messagefoundry.toml          # THIS instance's settings (environment + posture + store + API +
egress)
└─ requirements.txt              # pins the engine: messagefoundry==0.3.2
└─ .github/workflows/check.yml   # CI: install the pinned engine + run `messagefoundry check` on every
PR
└─ .vscode/settings.json         # points the VS Code extension at config/ + messages/
└─ .gitignore .gitattributes     # excludes stores, secrets, captures, venvs, caches
└─ README.md

```

Validate it immediately:

```

pip install -r requirements.txt
messagefoundry check --config config --messages messages/sets

```

`check` runs `validate` + `dry-run` against the synthetic fixture — the same gate your CI runs on every pull request. From here, your developers replace the starter feed with real Connections/Routers/Handlers, authored against the engine's public surface (`inbound` / `outbound` / `@router` / `@handler` / `Send` / `Message` / `MLLP` / `File` / `env` / `code_set` ...). They never touch engine code to do it.

5. Step 3 — Make it a *private* git repo

The scaffolded directory is ready to become your private repo:

```

git init
git add .
git commit -m "Initial MessageFoundry config (scaffolded)"
# then point it at your private host, e.g.:
git remote add origin git@github.com:your-org/mefor-config.git # a PRIVATE repo
git push -u origin main

```

Where it's stored — local or remote. The `git remote add` above points at a shared host, the right default for **HA (multiple engine hosts), a team, or off-machine backup**. For a single-box (non-HA) engine you can skip the remote and keep the repo local, adding one later. Any host works — a self-hosted **Forgejo/Gitea/GitLab**, a private repo on your standard host, or a **bare repo on a network share** for air-gapped sites. The IDE's **Set Up Version Control & Checks** and **Config Repo Storage Location** commands set and change this without the terminal. Full guidance: [VERSION-CONTROL.md](#).

What makes this private and safe:

- **Repo visibility.** Create the remote as **Private** on whatever host you standardize on. The engine imposes no requirement here — your config repo is yours, access-controlled by your git host's permissions (teams, SSO, branch protection).

- **No secrets, ever.** The scaffolded `.gitignore` already excludes `*.db`, `.env`, captures, and build craft. **Secrets — DB passwords, keys, credentials — are injected at runtime via `MEFOR_*` environment variables**, never written to the repo. The versioned `environments/<env>.toml` files hold only **non-secret** endpoints (hostnames, ports), which is exactly what you want diffable and reviewable.
- **No PHI in the repo.** Test fixtures are **synthetic only**. Real message bodies live in the engine's secured store at runtime, not in git.
- **CI as the gate.** The included `check.yml` installs the pinned engine and runs `messagefoundry check` on every PR. Turn on **branch protection** on `main` so changes land only through reviewed, check-passing pull requests — interface logic gets the same rigor as application code.
- **Branch → PR → review → merge** is the entire authoring workflow. There is no separate "engine change" path for developers, because there is no engine to change.

6. Step 4 — Configure each instance (settings + posture + secrets)

Each deployed instance gets its own `messagefoundry.toml` (the scaffolder generates a starting one) plus its `MEFOR_*` environment. Two things every instance must state:

- `[ai].environment` — a free-form name (`test`, `prod`, `poc`, ...) that selects `environments/<name>.toml`.
- **Security posture, explicit and decoupled from the name:** `[ai].data_class` (`synthetic` | `phi` — does this instance carry *real* PHI?) and `[ai].production` (is this a production tier?). Built-in names `dev` / `staging` / `prod` derive a sensible default posture; **any custom name must state posture explicitly** — the engine fails closed rather than guess.

Secrets and host-specific overrides come from the environment, e.g. `MEFOR_VALUE_<KEY>` for values used by `env("...")` in the graph, and `MEFOR_<SECTION>_<KEY>` for service settings. Precedence is **CLI flag > `MEFOR_*` env > `messagefoundry.toml` > built-in default**. Full reference: [CONFIGURATION.md](#).

7. Step 5 — Run / deploy an instance

```
messagefoundry serve --config config --env test --project-root C:\srv\mefor\my-config-repo
```

- `--project-root` (or `[environments].base_dir` in `messagefoundry.toml`) anchors `environments/<env>.toml` resolution to the repo root, so values resolve **regardless of the working directory**. This matters when the engine runs as a **Windows service under NSSM**, where the launch directory isn't your repo. (Omit it only when you always launch from the repo root.)
- The engine **binds `127.0.0.1` by default** and **requires authentication**. To expose a channel off-loopback, configure **native TLS** (API: `[api].tls_cert_file` / `tls_key_file` or a trusted upstream terminator; MLLP inbound: per-connection `tls = true`) — a non-loopback bind without TLS is **refused at startup**. See [DEPLOYMENT.md](#).
- For production, run the engine as a **Windows service via NSSM** — see [SERVICE.md](#).

- For **multi-node high availability** (active-passive failover of a single instance), see [CLUSTERING.md](#) — note it requires an operator-provided **floating VIP / L4 load balancer** (a TCP-connect health check per inbound listener port) so senders follow the primary across a failover. MEFOR ships the clustering + the `/cluster/*` health-check endpoints, but **not** the load balancer itself; single-node deployments need none of this.

Launching the admin console (in a browser)

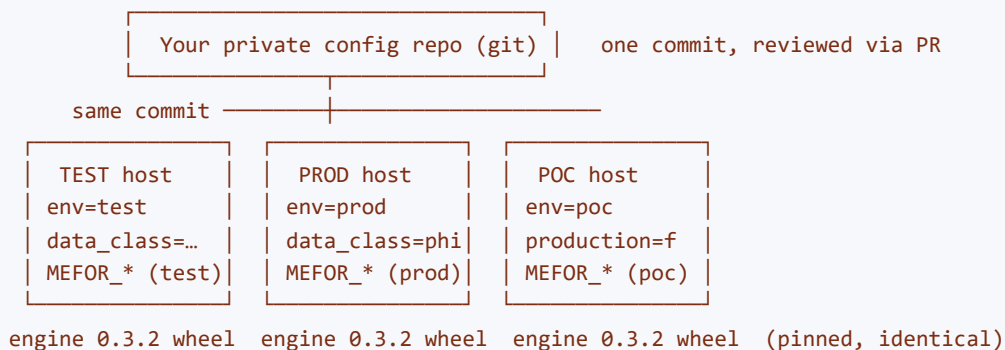
Operators monitor and run an instance from the **browser web console** served same-origin at `/ui`. It ships as a separate wheel (`messagefoundry-webconsole`) on its **own version line** — deliberately *not* lockstep with the engine — that the engine mounts in-process. The pair is checked at startup against the engine's UI-seam version and a mismatched pair is **refused**, so install the console release built for the engine version you pinned. The console is **on by default**, so on a local instance installing it is all you need:

```
pip install "messagefoundry-webconsole==0.2.15" # the /ui web console, into the same venv
# then (re)start the engine – there is no switch to turn on. To turn the console OFF, set
# [security].serve_web_console = false (the old [api].serve_ui spelling is refused at config load)
```

Browse to the engine's `/ui` (`http://127.0.0.1:8765/ui` by default — typically the boot-start **service**) and sign in; nothing else is needed for the local case. Off-loopback the console is **opt-in**: an exposed instance serves `/ui` only when `[security].serve_web_console = true` is set explicitly — a default-on console on an exposed bind quietly degrades to the JSON API with a warning — and it additionally requires TLS, plus `[security].web_console_public_address` behind a declared TLS-terminating proxy (see [REMOTE-CONSOLE.md](#)). The former PySide6 desktop console was retired in favour of this browser console (BACKLOG #103); PySide6 now backs only the standalone test harness (`pip install "messagefoundry[harness]"`).

8. Multiple instances from one repo (Test, Production, POC...)

This is the payoff of the model: **the same reviewed commit of your config repo is deployed to every instance**. You do **not** maintain per-environment branches. Each host differs only in its `messagefoundry.toml` (environment name + posture) and its `MEFOR_*` environment:



Promotion = merge to `main` → deploy that commit everywhere. A Test instance resolves `environments/test.toml`; Prod resolves `environments/prod.toml`; a Test instance can never accidentally read Prod values. (Each instance is otherwise an independent engine with its own store; for multi-node high availability of a single instance, see [CLUSTERING.md](#).)

9. Why integration developers can't modify the engine

Three layers reinforce the boundary:

1. **Packaging.** The engine is an installed, **non-editable** wheel. Developers work exclusively in the config repo; the engine source isn't in their working tree.
2. **Version pinning.** `requirements.txt` pins an exact engine version; any change to it is a visible, reviewable PR — and CI re-runs `check` against the new version before it can merge.
3. **(Optional) OS enforcement.** Operators can make the venv's `site-packages` read-only (Windows ACL) so the installed engine can't be edited in place even by mistake.

There is simply no developer workflow that routes through engine source — by construction.

10. Upgrading the engine

1. Bump the pin in `requirements.txt` (e.g. `messagefoundry==X.Y.Z`) on a branch.
2. `pip install -r requirements.txt` and run `messagefoundry check` locally; open a PR — CI re-validates your whole config against the new engine.
3. Merge, and roll the new commit to Test first, then Production. Because everything is pinned and your config is gated by `check`, upgrades are deliberate and reversible (pin back). The full upgrade/rollback runbook is in [EARLY-ADOPTER-GUIDE.md](#) §13.

Where to go next

Topic	Reference
Staged install-to-production rollout, hardening, DR	EARLY-ADOPTER-GUIDE.md
The consumer deployment model (rationale)	ADR 0017
Service settings / environments	CONFIGURATION.md
Connections / the graph / <code>connections.toml</code>	CONNECTIONS.md
Windows service install	SERVICE.md
Network exposure / TLS	DEPLOYMENT.md

Topic	Reference
Security / auth / RBAC / audit	SECURITY.md
PHI handling / encryption	PHI.md
Multi-node high availability (needs a floating VIP / L4 LB)	CLUSTERING.md

Bottom line

Install the pinned engine into a venv; run `messagefoundry init` to scaffold a private config repo; push it to your private git host with branch protection + the included CI check; deploy as many instances as you need from that one repo, each with its own environment name, posture, and secrets. Your developers get a clean, reviewable, code-first authoring experience, and the engine stays a fixed, auditable dependency they never edit.

© 2026 **MEFOR-ORG**, a non-profit · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated July 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MEFOR-ORG. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.